

Instructor perspectives on an embedded coding course in the civil engineering curriculum

Wissam Kontar¹ and Kaycie Lane^{1,*}

¹University of Nebraska-Lincoln, Lincoln, Nebraska, USA

*Corresponding author: kaycie.lane@unl.edu

Abstract

This study reports instructor perspectives on the implementation of a Python-based data analysis and coding class into the civil engineering curriculum at the University of Nebraska-Lincoln over two years, taught by three different instructors. Using instructor weekly meetings and observations over two years, this study reports improvements made between the first and second years of implementation of Civil Engineering Analysis II (CIVE 202), lists proposed improvements and ideas for future implementation, documents challenges with implementation and benefits to students derived from the innovative design of this course. In the effort to ensure the utility of CIVE 202 skills to student's future careers as engineers, improvements have included using client prompts and real-world data analysis problems, integrating multiple civil engineering subdisciplines (transportation, environmental, etc.) and changing lab activities to be more about "proof of learning" from individual students to ensure all learning outcomes are met. Continuing challenges include balancing individual and group work, supporting the instructors teaching the course and designing and integrating activities and assignments that promote student engagement. Noted benefits to students based on the course design include: teaching students how to translate a client prompt into a series of analysis steps, learning how to troubleshoot and an understanding of how AI can be used in the troubleshooting process, destigmatization that coding is "hard", developed awareness of algorithmic bias and its implications and the importance of translating coding products into reports for clients.

Keywords: Civil engineering, coding skills, data analysis, course development, faculty paper

Introduction

Integrating programming languages such as Python and MATLAB into engineering curriculum has followed a slow trajectory over the past ten years, with most engineering curricula simply switching from requiring C-based languages to other languages taught within the computer science departments at the university [1], [2], [3], [4], [5]. Adding coding and data analysis skills to engineering curricula is both timely and relevant [3]; however, the applicability of these programming courses to student's future engineering careers is questionable, especially when very few engineering programs have a faculty member within the department teaching the course. It is more common to outsource coding courses to other departments either for expediency or due to a lack of teaching capacity within the department (teachers trained in Python data analysis, available time in teaching schedules, etc.). However, previous literature

shows that when courses are taught outside of a major department (i.e. Calculus taught by the Math Department and not the Civil Engineering department), students struggle to connect fundamental concepts and theorems to engineering applications [6], [7]. Literature also suggests that project-based learning and application-based learning also have real benefits to students as opposed to assigning simple textbook problem sets for assessment [8], [9], [10], [11], [12]. Therefore, a project- and application-based Python course, taught within the Civil Engineering department potentially provides students with the most benefit to their future careers.

At the University of Nebraska-Lincoln (UNL), Civil Engineering Analysis II (CIVE 202) is a sophomore-level course designed to (1) teach civil engineering majors Python-based data analysis skills and (2) keep civil engineering students connected to professors within the major as part of the majors new Design Spine curriculum [13]. CIVE 202 is the fourth in a sequence of eight classes in the Design Spine, with one class per semester every semester for students in the civil engineering major and follows an Excel-based course entitled Civil Engineering Analysis I. The Design Spine was implemented first in Fall 2022 at UNL with the 100-level courses, in 2023 with the 200-level courses and 2024 with the 300-level courses (a figure of the structure can be seen in Figure 1). The Design Spine is also focused on propagating software skills such as Excel and Python through the curriculum by building the skills in the sophomore year so that instructors of 300-level and 400-level courses can easily integrate these skills into design-based courses to provide engineering graduates with skills that will set them above competitors when finding engineering internships and jobs [13]. The long-term goal is to integrate Python skills from CIVE 202 into 300-level and 400-level courses so that coding is not an afterthought in engineering curriculum but an integral part of how future engineers are thinking about the design process.

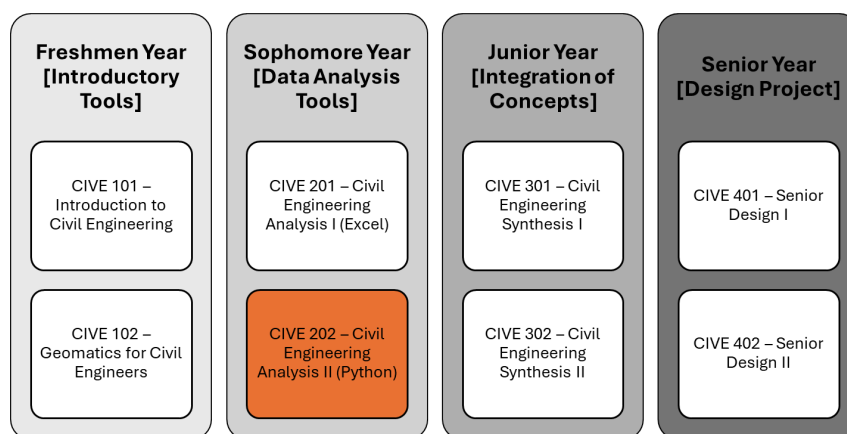


Figure 1: The Design Spine structure at the University of Nebraska-Lincoln

CIVE 202 is designed to focus on the process of data analysis, from data organization to analysis, visualization, interpretation and finally the integration of the proceeding four steps so that students leave the course with an understanding of how data analysis is used by professional engineers. While Python is the coding language used to teach data analysis, the emphasis of the

class is on the communication of data analysis results to a real-world client rather than on the syntax of code [14]. The full course design process was documented in Lane and Hawkins, 2024, including project-based structure, types of assignments, assessment strategies, syllabus policies, learning objectives and suggested improvements after the first year of implementation [14]. The class focuses on using real-world client prompts and a project-based structure, taught as a combined lecture-lab on one day of the week, with an AI policy to set clear guidelines on AI usage, and involves big data sets from across several civil engineering subdisciplines (geotechnical, structural, transportation, environmental, etc.).

The objective of this study was to document instructor perspectives on the initial two years of CIVE 202 implementation at the University of Nebraska-Lincoln. We have highlighted improvements made, future proposed improvements, challenges to implementation and benefits to students. A forthcoming study will document student perspectives on the first three years of CIVE 202 implementation to complement this work.

Methods

CIVE 202 has been taught twice to date in the spring semester by three different instructors. The structure of the course was developed in Fall 2023 in the Learning by Design workshop offered by the Engineering and Computing Education Core (ECEC) at UNL. The course was first implemented in Spring 2024 and for a second time in Spring 2025 with the majority of the class structure remaining the same to allow for comparisons between each year. Instructors teaching the course met weekly, either in person or on Teams to discuss challenges, changes to activities or materials, and general observations of how the class was progressing. The instructors kept a record of changes and improvements made between Year 1 and Year 2, noted desired improvements for future iterations of the course, challenges faced when implementing the new curriculum and benefits to the students. This paper focuses on the instructor perspective of implementation over 2-years; forthcoming papers will focus on student feedback once an IRB approval has been completed to use student survey responses.

Integrated and Proposed Improvements

Improvements from Year 1 to Year 2

After implementing CIVE 202 for the first time in Spring 2024, we made several small modifications to the structure and delivery of the class prior to teaching in Spring 2025. First, more in class activities were added to the course structure, particularly in lecture where the class had been primarily didactic lecturing using presentations. In Spring 2024, we noted students enjoyed activities in lecture, finding utility from peer reviews and time to work with their teams on activities that supported their projects. As a result, we added a coding challenge, an activity on reproducibility, and generally more time for students to explore and research topics on their own. In labs, we changed templates from a “fill in the blank” based on the instructor walking through an activity to a “proof of learning” template where students had to demonstrate they

could alter code taught by the instructor to complete a task. We integrated most of the activities in Project #3-5 once coding skills and the class structure had been established in the first two projects and saw an increase in students meeting learning objectives. For example, in Spring 2024, many times lab submissions did not show creativity or individual thought, and we were therefore unable to reliably evaluate whether students had met the learning objectives. However, in Spring 2025, with the “proof of learning” template, we were better able to evaluate whether individual students were achieving each learning objective and better identify areas to improve in the future.

Second, in Spring 2025 we added more project options and increased the relevance and quality of data sets used for the projects. For a description of the projects in depth, please reference Hawkins and Lane, 2024, which details the projects developed for CIVE 202 in Spring 2024. First, Project #1 was updated to have cleaner data, more concise templates for students and a greater emphasis on formatting tables, similar to using pivot tables in Excel, a skill most students had learned in CIVE 201 in the Design Spine (Figure 1). Project #2 was relatively unchanged but statics problems were altered to be more relevant to concepts students were learning in statics at the time of the project. Project #3 was updated to have cleaner data, better time series data to plot and more plotting options to allow students to get creative with their plot customizations. Project #4 still focused on bias in code, but shifted from the Joint Monitoring Programme’s sanitation dataset to the National Risk Index (NRI) data set in the United States. While both allowed students to examine infrastructure concerns, the NRI dataset was more directly applicable to student’s interests, allowed for more customization of code and solutions to the project prompt, had lots of metadata and documentation for students to explore and generally was more closely aligned to what sophomore students have been exposed to in their civil engineering careers. Project #5 included the addition of a real world client for an air quality monitoring project, changes to the transportation projects to better align with Project #3 datasets, changes to infrastructure projects to better align with Project #4 datasets and more flexibility on whether students wanted to work in groups or individually, and who students wanted to work with. We also tested the challenge of individual vs. group work by trying Project #1 as an individual project in both Lincoln and Omaha and by trying Project #3 as an individual project in Omaha only.

Proposed Future Improvements

In line with the improvements made to CIVE 202 from Year 1 to Year 2, we propose to continue changing didactic lectures into integrating activities, to change lab templates from “fill in the blank” to “proof of learning” to make sure students learn concepts individually, and to continue striving to find a balance between individual and group work. The challenge associated with the balance between individual and group work often comes down to practicality for at least the Lincoln location of this course: the class size has grown to require to lab sections and has over 80 students with only one teaching assistant and grading takes up a substantial amount of time, especially because this class was designed to have many small assignments and very involved

reports. While groupwork does help to provide some grading relief, it does make it very difficult to verify individual student learning. The utility of group work is almost always anecdotally praised as providing students with teamwork skills to meet ABET requirements; however, not all literature agrees that teamwork is 100% effective, especially when the burden falls on one student to complete the project due to laziness, disengagement or miscommunication by other group members [13], [14], [15], leading to dissatisfaction and a loss of valuable learning time. We have observed that team work has helped many students balance the workload of the projects and helps many students to get peer-to-peer feedback and learning experiences that improve their understanding. But we have also observed groups where one person does all the work (particularly assigning one person to be the “coder” which defeats the purpose of teaching all students Python), students do not communicate with each other, and general procrastination makes teamwork more stressful than individual work. Balancing individual and teamwork will therefore be an ongoing challenge to manage in CIVE 202.

Another area we propose to focus on is troubleshooting. In previous iterations of the course, we spent about 20 minutes of one of the first lectures in Project #1 on ideas for troubleshooting. However, since most students had not yet been using Jupyter Notebooks and Python for very long in the course at this point, we found this lecture was more academic than practical for the majority of students. One easy change would be to add this lecture content later in Project #1 and to turn it into an activity: for example, by creating a template in Python that is purposefully “broken” with erroneous code and walk through lab to show students different ways to troubleshoot. One of the professors who taught lab also frequently used troubleshooting as a teaching tool in lab when students had questions. Because this professor did not have lab teaching support, this professor did not correct student’s code when students had questions; rather, this professor asked students to identify the error and choose a troubleshooting tool, then walked the student through how to find the correction themselves. This helped this professor combat the challenge of not having a teaching assistant while also teaching students how to troubleshoot. This technique is one that we plan to implement more systematically in the future across both teaching locations and is a skill we plan to teach teaching assistants so students do not default to asking how to fix an error, but learn how to troubleshoot on their own. Learning how to troubleshoot is a more critical skill as an engineer than knowing how to write perfect Python code; we therefore want to emphasize how to troubleshoot, verify the validity of solutions found and to implement those solutions in future iterations of the course.

In line with the changes to Project #3 and Project #4 made in Year 2, we propose to continue updating the projects in the course to cover the 5 subdisciplines of civil and environmental engineering topics offered in the department (geotechnical, water and resources, structural, transportation and environmental); specifically, we want to make sure each topic is covered by a project in some capacity with real world, useful applications that students will see in summer internships and future 300-level courses. The class is still skewed towards the past professor’s specialties’ of environmental and transportation engineering; we have attempted to engage with

other professors in the department to develop new projects and partnerships but have found this difficult due to heavy teaching loads or other commitments that limit the effectiveness of engagement if even available. We have considered having other professors act as the “client” in a project to help make sure we are developing Python skills that will be relevant to the professor’s classes in the 300-level and 400-level discipline specific courses but again run into the challenge all professors are facing: time constraints. An alternative strategy could be to have new professors teach this course so that new projects get developed in-line with each professor’s sub-discipline. However, one challenge we have repeatedly seen within our department and in other departments is the “copy and paste” method of teaching a class, wherein a professor uses past materials from another professor without changes. While this method helps many overloaded or new professors get started with a new class, it results in a propagation of errors much like in coding, where no new changes are integrated and old ways of teaching that may have lost effectiveness continue to be used when new techniques may provide better opportunities for student learning. We continue to explore opportunities to integrate more civil engineering subdisciplines.

All of the previously mentioned improvements signal a bigger change that needs to be made in order for CIVE 202 and similar types of classes to be successfully integrated within an engineering department: the class needs departmental support in upper-level courses. Furthermore, since CIVE 202 is part of the new Design Spine at UNL, the skills learned in CIVE 202 need to be used in 300-level and 400-level classes. For tools like Excel, the integration is often obvious because many professors use Excel in their daily lives. However, very few professors in the UNL Civil and Environmental Engineering department currently code in Python; those who do use it for advanced research modeling, with few professors having ever used Python in their undergraduate careers which results in challenges with how to integrate a completely new language into classes many professors have been teaching for 5-20 years. The ethos of the Design Spine is to (1) keep civil engineering students engaged with civil engineering professors and applications, but more importantly for this study, (2) to integrate critical software and technical skills across the curriculum. To this end, Python lessons or components do need to be integrated into 300-level and 400-level classes across the curriculum (there are over 20 classes offered where this could happen). Next steps for the instructors of CIVE 202 will be to work with 3-5 professors from the 300-level classes who are willing to become “early adopters” of Python modules. Recruitment efforts are currently underway and will continue in the coming semesters as a first step to better curriculum integration of CIVE 202 concepts.

Challenges to Successful Implementation

One of the biggest challenges to implementing the course was utilizing the lecture and lab time most effectively, as noted in a previous paper on the structure of this course (Hawkins and Lane, 2024). Only having contact with students once a week for a 50-minute lecture and a 2 hour lab made it difficult to go into any depth on key topics. For example, students enjoyed engaging with examples of algorithmic bias, generating discussion in the class that had to be cut short due

to the limited time available and the need to move on to lab activities. We attempted to allow flexibility into the schedule by letting lecture activities carry into lab since the lab and lecture were back to back in Omaha, however, with two sections in Lincoln, this was not possible. The other challenge associated with only meeting once a week was that students tend to disengage from the material and only focus on submitting the activities right before class starts, which means they are not continuously engaging with material and practicing outside of class.

Our second challenge follows from the concern with student disengagement outside of class times. We found that many students do not “follow-up” on their own learning and procrastinate on the small assigned activities we designed to reinforce lecture and lab content. While not uncommon, with the format of this course, we found it difficult to assign enough assignments to reinforce learning objectives, but to also balance our assignments with the other challenging course students are taking in their sophomore year. We found that there were about 91 students across both campuses (5 students total) who stopped engaging with the material in the course after Project #1. We saw students getting left behind early and being unable to catch up. When trying to engage with these students to close the gap (email outreach, Canvas announcements, office hours, etc.) we found (1 dropped out, 1 failed, and 3 managed through). One possible solution to this concern would be to change how assignments and time outside of class are incentivized to students. Currently, many of the assignments were small activities worth minimal points, but designed to aid in the final products required in each Project. Changing the point values associated with assignments, the amount of work involved in the assignments or the due dates of the assignments to earlier in the week all could provide viable options to change student engagement patterns.

We also noted that many students start the course with a “mental block” or preconceived notion about what coding is or their ability to code. This often manifested as hesitation to engage with programming tasks in class, fear of making errors, or a belief that coding requires an innate aptitude rather than practice. To address this, a critical step is emphasizing that coding is not about memorizing syntax but about solving a problem through logical thinking and stepwise iteration. It is essential here to build individual student confidence early on in the semester. One aspect of this is introducing low-stakes, structured activities in the first few weeks of the semester to normalize failing and exploration in coding. Further, we provide individual feedback and learning opportunity, which saw several students shift from apprehension of memorization into curiosity. For instance, re-writing the same code with different syntax or style. The mental barrier was a significant challenge and key element throughout the course.

As mentioned in the improvements from Year 1 to Year 2, one of the big challenges still remaining in CIVE 202 is to balance individual and group work. To iterate the challenges presented in the “Improvements from Year 1 to Year 2” section, the following questions need to be addressed:

2025 ASEE Midwest Section Conference

1. How to ensure all learning outcomes are met by all students when group work is assigned? How can we change assignment structures and rubrics to ensure we can evaluate all students?
2. How can we ensure teamwork is mutually beneficial to group members and not a burden on one person?
3. How can we provide more individualized feedback and support for students who struggle early in the class so they do not fall and stay behind?

Finally, as alluded to in the “Future Improvements”, there needs to be a concerted effort to support CIVE 202 in 300-level and 400-level courses. Furthermore, there needs to be stronger support from the faculty, department and college of engineering to support the instructors of CIVE 202. For example, in Spring 2024, both sections of the course had teaching assistant support; however, in Spring 2025, only the Lincoln section had a teaching assistant due to a lack of funding for graduate teaching assistants, a lack of graduate students with Python knowledge, and scheduling conflicts. Without teaching support, it became difficult for the Omaha instructor to adequately teach the lab portion of the class and debug and answer all students questions in a timely manner, leading to challenges covering all necessary material, students disengaging from lab while the instructor was working with other students and difficulties getting grades back in a timely manner. In addition to not having trained teaching assistants, there are also few instructors who know the Python coding language; in fact, one of the three instructors who taught CIVE 202 in the past 2 years did not know Python prior to teaching CIVE 202. This results in a reliance on specific people in the faculty “carrying” the class forward which is not a sustainable long-term curriculum strategy as teaching schedules need to be flexible and change as faculty retire, change positions or go on faculty development leave.

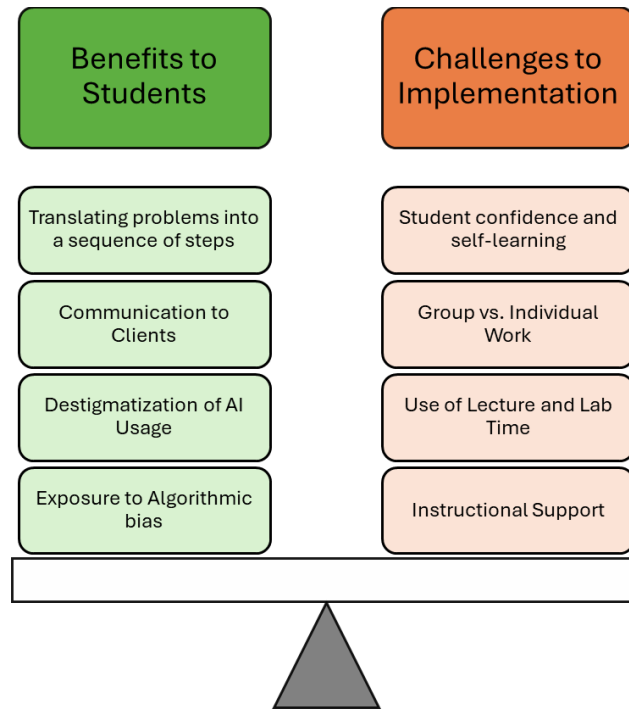


Figure 2: Many of the instructional design challenges are offset by the benefits to students, making future iterations and revisions necessary to improve the course.

Benefits to Students

There are several benefits to students we documented over the course of two years of implementation: (1) coding teaches students to think about translating a client prompt or engineering problem into a series of analysis steps, (2) learning how to troubleshoot is a critical engineering skill, (3) destigmatization that AI tools are “cheating” and an understanding of how AI can be used in the troubleshooting process, (4) destigmatization that coding is “hard”, (5) developed awareness of algorithmic bias and its implications and (6) the importance of translating coding products into reports for clients. Each of these benefits are explained in detail below.

Translating Client Prompts into Analysis Steps

One of the most important benefits observed was students’ development of the ability to translate a client prompt or engineering problem into a structured series of analysis steps. Rather than following a predetermined set of instructions. Students were required to independently interpret the problem and how to approach it using python. A shift from passive to active problem solving, with python as the “linguistic” tool of communication. Because python was a new language for most students, they had to think carefully about the logical structure of their solution. In this way, coding functioned similarly to learning a foreign language. For example, in working with functions, students had to define variables, apply algebraic reasoning, and implement solutions for a problem they are very much familiar with (prior course taken) but in a completely new

format. By repeatedly engaging with such problems, students start working as practicing engineers, meaning they identify key complements of a problem, design a strategy, and communicate their reasoning (here through python and not language).

Troubleshooting Skills and AI Destigmatization

One benefit to students was the integration of troubleshooting skills in labs and activities, enabling students to become familiar with independent problem solving. Learning which terms to put into internet search engines and AI generators to get the “correct” answer is a skill in itself that takes practice. Furthermore, translating the results into code is also a skill; many students copy and paste the results of a search into their code and do not know how to change solutions from the internet to work with their code. When students learn how to use internet resources properly for troubleshooting, many students have a “breakthrough” moment and begin to see coding as a puzzle to solve rather than a series of functions on screen and begin to understand what each piece of code is doing. Knowing how to successfully troubleshoot sets apart students who do well in from CIVE 202 from students who do not engage with the material and do not understand the rationale behind coding functions.

Because AI generators are prevalent, CIVE 202 was designed with the following two components meant to destigmatize AI usage for troubleshooting and to reinforce the need to understand what each piece of code is doing: (1) an AI policy that specifically tells students AI is only allowed for troubleshooting and if AI is used it must be cited in (2) an annotated code document (ACD), which is a table showing the Python code on the left and a detailed explanation of what the code is performing (with references if code was sourced online) on the right. Students did use ChatGPT and other AI generators during labs; in fact, instructors often used AI generators themselves as a teaching tool when troubleshooting one-on-one with students, focusing on how to use and adapt the code provided by the AI. The class also has a resubmission policy, wherein students can submit their work as many times as they want to receive feedback before the hard deadline. When students made use of this policy, instructors were able to catch uncited code in the ACD and request revisions. One instructor noted that when providing this comment, the students who received the comment corrected the issue promptly and did not miss a citation thereafter. As a result of these policies, we noted that while AI generators were used, templates in labs were often referenced more than online help and that having an AI policy and an ACD component to each project helped to destigmatize the use of AI as a “cheating” tool and turn it into a troubleshooting tool instead.

Destigmatization of Coding as “Hard”

CIVE 202 helped break the common perception that coding is inherently difficult or only for advanced students. Many undergraduates realized they could perform tasks often associated with graduate-level work. The course emphasized that coding is not just about syntax but about holistic structured thinking. Embedding this in civil engineering curriculum reinforced that coding is a practical tool for problem solving not a standalone or inaccessible skill.

Exposure to Importance of Algorithmic Bias

The introduction of algorithmic bias was one of the most impactful parts of the course. Many students were unaware that algorithms used by platforms like Google or social media sites tailor content, advertisements, and even search results based on personal data such as location and browsing history, or even lead to misjudgments in data analysis. This realization sparked some of the most thoughtful and engaged discussions of the semester. Students analyzed real datasets and explored how algorithmic decisions can reinforce social and structural inequalities. Students began to understand the broader implications of the code they write. This was particularly important given the increasing integration of AI tools in engineering. While CIVE 202 allows the use of AI for troubleshooting, the discussion on algorithmic bias reinforced that misuse of AI can lead to significant harm. This topic helped students begin to develop a sense of ethical responsibility, showing them that engineers not only solve problems but also shape how technology affects society.

Translating Coding Products for Clients

One of the most difficult yet valuable skills emphasized in CIVE 202 was the ability to explain coding processes and results to an assigned client. Students were expected not only to generate code that met the technical requirements of a project, but also to communicate their methods, logic, and conclusions clearly in written form. This component most closely mirrors what civil engineers are expected to do in professional practice, which is to translate technical work into deliverables that clients, stakeholders, or non-technical colleagues can understand and act upon.

While learning Python syntax was important, anecdotal feedback from both professors and industry members strongly indicated that the ability to explain code in written reports was more valuable than code-writing alone, especially at the undergraduate level. In fact, many professionals emphasized that communication is often the limiting factor in a project's success, not the technical analysis itself. Embedding this skill into an early-level course gave students realistic practice in bridging the gap between problem solving and communication.

However, this task proved to be one of the most challenging for students. Many had never been asked to explain their thinking in such detail or to justify their use of code in writing. Translating their technical process into client-ready summaries required them to consider audience, relevance, and clarity. Through repeated assignments throughout the course, students gradually improved their ability to contextualize their work and communicate engineering insight through narrative, tables, and annotated code. We believe this skillset is one of the most directly transferable to the consulting roles many of our students will enter after graduation.

References

- [1] D. G. Balreira, T. L. T. D. Silveira, and J. A. Wickboldt, "Investigating the impact of adopting Python and C languages for introductory engineering programming courses," *Comput. Appl. Eng. Educ.*, vol. 31, no. 1, pp. 47–62, Jan. 2023, doi: 10.1002/cae.22570.

- [2] Q. Cao, L. H. I. Lim, V. Dale, and N. Tasler, “EXPERIENCES IN PYTHON PROGRAMMING LABORATORY FOR CIVIL ENGINEERING STUDENTS WITH ONLINE COLLABORATIVE PROGRAMMING PLATFORM,” presented at the 14th annual International Conference of Education, Research and Innovation, Online Conference, Nov. 2021, pp. 5784–5791. doi: 10.21125/iceri.2021.1305.
- [3] S. Grajdura and D. Niemeier, “State of Programming and Data Science Preparation in Civil Engineering Undergraduate Curricula,” *J. Civ. Eng. Educ.*, vol. 149, no. 2, p. 04022010, Apr. 2023, doi: 10.1061/(ASCE)EI.2643-9115.0000076.
- [4] G. List, “Introducing Civil Engineering Analysis Through Programming,” in *2007 Annual Conference & Exposition Proceedings*, Honolulu, Hawaii: ASEE Conferences, June 2007, p. 12.961.1-12.961.10. doi: 10.18260/1-2--2303.
- [5] P. Von Lockette, “Algorithmic Thinking And Matlab In Computational Materials Science,” in *2006 Annual Conference & Exposition Proceedings*, Chicago, Illinois: ASEE Conferences, June 2006, p. 11.168.1-11.168.13. doi: 10.18260/1-2--725.
- [6] B. Faulkner, N. Johnson-Glauch, D. San Choi, and G. L. Herman, “When am I ever going to use this? An investigation of the calculus content of core engineering courses,” *J. Eng. Educ.*, vol. 109, no. 3, pp. 402–423, July 2020, doi: 10.1002/jee.20344.
- [7] J. Quintanilla, N. D’Souza, J. Liu, and R. Mirshams, “Integration Of Engineering Concepts In Freshman Calculus,” in *2007 Annual Conference & Exposition Proceedings*, Honolulu, Hawaii: ASEE Conferences, June 2007, p. 12.938.1-12.938.8. doi: 10.18260/1-2--2522.
- [8] S. Lavado-Anguera, P.-J. Velasco-Quintana, and M.-J. Terrón-López, “Project-Based Learning (PBL) as an Experiential Pedagogical Methodology in Engineering Education: A Review of the Literature,” *Educ. Sci.*, vol. 14, no. 6, p. 617, June 2024, doi: 10.3390/educsci14060617.
- [9] A. Karimi and R. Manteufel, “Alternatives to Textbook Homework Assignments,” in *2018 ASEE Annual Conference & Exposition Proceedings*, Salt Lake City, Utah: ASEE Conferences, June 2018, p. 29768. doi: 10.18260/1-2--29768.
- [10] J. E. Mills and D. F. Treagust, “ENGINEERING EDUCATION – IS PROBLEM-BASED OR PROJECT-BASED LEARNING THE ANSWER?”.
- [11] M. K. Noordin, A. N. Nasir, D. F. Ali, and M. S. Nordin, “Problem-Based Learning (PBL) and Project-Based Learning (PjBL) in engineering education: a comparison,” 2011.
- [12] J. Ristroph, “An Alternative Paradigm For Engineering Homework: The Case Of Engineering Economics,” in *2006 Annual Conference & Exposition Proceedings*, Chicago, Illinois: ASEE Conferences, June 2006, p. 11.171.1-11.171.16. doi: 10.18260/1-2--431.
- [13] E. Jones, “Designing a New Civil Engineering Curriculum to Prepare Tomorrow’s Engineer,” in *2024 ASEE Annual Conference & Exposition Proceedings*, Portland, Oregon: ASEE Conferences, June 2024, p. 47135. doi: 10.18260/1-2--47135.
- [14] K. Lane and J. Hawkins, “Design and Implementation of a Programming Course Embedded in Civil Engineering Curriculum,” in *2024 ASEE Midwest Section Conference Proceedings*, Lawrence, KS: ASEE Conferences, Sept. 2024, p. 49364. doi: 10.18260/1-2-115-49364.

First Author Biography

2025 ASEE Midwest Section Conference

Dr. Wissam Kontar is an Assistant Professor of Civil and Environmental Engineering at the University of Nebraska–Lincoln. His research focuses on understanding, modeling, and improving complex transportation systems. He has authored numerous works spanning transportation science, artificial intelligence, cognitive engineering, and control theory. His research is supported by the U.S. Department of Transportation and the National Science Foundation, and has been recognized with three Best Paper Awards from TRB and IEEE, as well as the 2022 Rising Stars Award in Cyber-Physical Systems. His teaching portfolio emphasizes theoretical and computational tools with applications across engineering domains, particularly in transportation.

Second Author Biography

Dr. Kaycie Lane is an assistant professor of practice at the University of Nebraska-Lincoln, teaching in the civil and environmental engineering department. Her technical expertise centers around drinking water and wastewater management and assessment, with over seven years of experience performing data analysis on big data sets from the water, sanitation and hygiene (WaSH) field in the R statistical programming language course. As a teacher, she emphasizes application-based learning through project-driven courses, the application of the engineering design process to unique scenarios and the addition of socio-technical thinking into engineering education. She teaches a wide variety of classes within the environmental engineering curriculum including Mass and Energy Balances, Environmental Microbiology and Water and Wastewater Treatment Design and in the broader civil engineering curriculum on the topics of data analysis and sustainability.